

ROS2 from Zero to Hero

Dr. Kitti Thamrongaphichartkul

Research Fellow

Nanyang Technological University

8 Oct 2026



github.com/kittinook/ros2_tutorial

Learn ROS 2 with a Mars rover

The whole course in one deck. For every mission: what you'll learn, the picture of what you'll build, how to run it, and a checklist.

13 missions · ROS 2 Jazzy or newer · github.com/kittinook/ros2_tutorial

What you'll learn

#	Mission	You learn
0	Landing	workspace, build, launch, teleop, remapping
1	Telemetry	nodes, topics, messages
2	Manual Drive	publishing <code>Twist</code> , speeds and angles
3	Mission Control	services: request and response
4	Survey Square	a package and a Python publisher
5	Waypoints	subscribers, <code>Odometry</code> , closed-loop control
6	Sample Hunter	camera and laser, service clients in code
7	Rover Fleet	namespaces, parameters, launch files
8	Boss: Power Crisis	all of the above, state machines, battery
9	Arm Check	joints, <code>JointState</code> , <code>SetBool</code> grippers, tf2
10	Frames	tf2 in code, inverse kinematics
11	Drill & Stow	actions: goal, feedback, cancel; pick and place
12	Boss: Meteorite Recovery	mobile manipulation, two arms

ROS 2 official / Tutorial docs

- About ROS 2: docs.ros.org/en/lyrical/About-ROS.html
- Install ROS 2: docs.ros.org/en/lyrical/Get-Started/Installation.html
- Tutorial: docs.ros.org/en/lyrical/ROS-Framework.html
- https://github.com/kittinook/ros2_tutorial/blob/main/ARCHITECTURE.md
- Concept: https://github.com/kittinook/ros2_tutorial/blob/main/CONCEPTS.md

Example

- Tele-operation System: https://github.com/synergylab-ntu/tele_ros2
- Haption: https://github.com/synergylab-ntu/haption_ros2
- Kinova Controller: https://github.com/synergylab-ntu/kinova_controller_pkgs
- Phasespace: https://github.com/synergylab-ntu/phasespace_ros2

Set up once

```
source /opt/ros/jazzy/setup.bash           # or your distro: humble, kilted, rolling ...
sudo apt install python3-pygame python3-numpy ros-$ROS_DISTRO-teleop-twist-keyboard
git clone <url of this repo> ~/mars_rover
cd ~/mars_rover
colcon build --symlink-install
source install/setup.bash

echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc           # so every new
echo "source ~/mars_rover/install/setup.bash" >> ~/.bashrc     # terminal is ready
echo "export ROS_DOMAIN_ID=<your number>" >> ~/.bashrc         # in a classroom
```

Every new terminal needs `source` . Forgetting it is the most common cause of "Package not found".

How every mission works

```
# terminal 1: start the mission (the simulator window + mission control)
ros2 launch mission_control mission.launch.py mission:=<number>

# terminal 2, 3, ...: do the work
```

- The **panel** on the right lists the objectives and ticks them by itself
- Finish them all and you get up to **3 stars** for speed. Coding missions start the clock when the rover first moves
- Coding missions check that **your own node** drives: teleop or `ros2 topic pub` costs stars
- Some missions can **fail** (battery flat, drill broken). Close the window and launch again. `ros2 run mission_control progress` shows your best stars

Part 1: the rover

Missions 0 to 8: the four ways ROS 2 programs talk, then your own nodes.

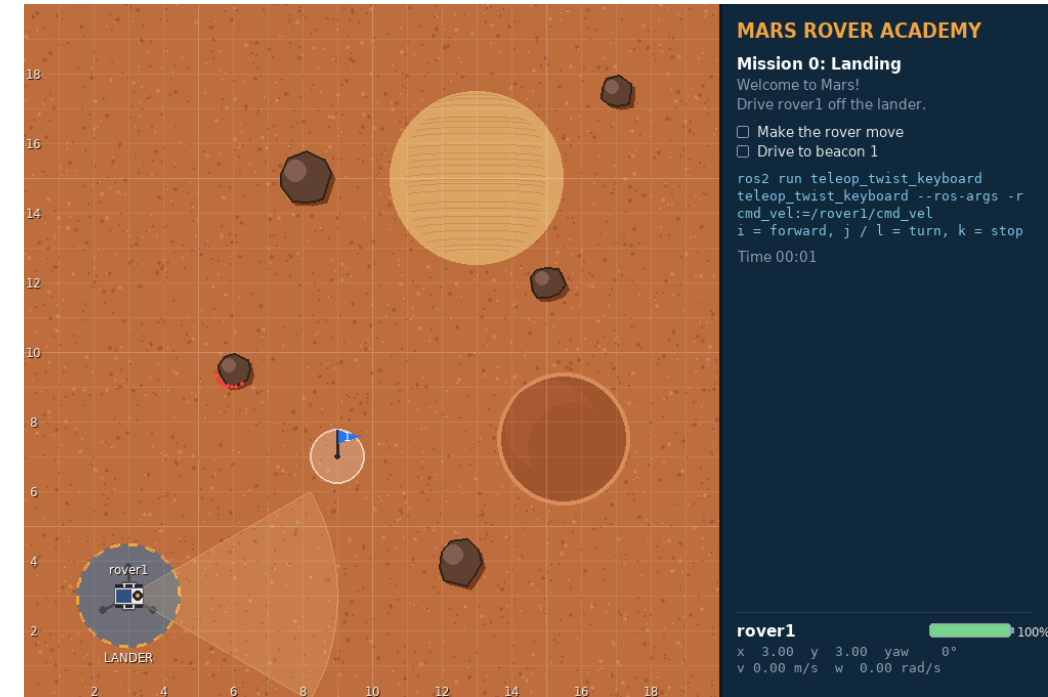
Mission 0: Landing

You'll learn: what a workspace and a package are,
`colcon build`, `source`, `ros2 launch`, `ros2 run`,
remapping a topic

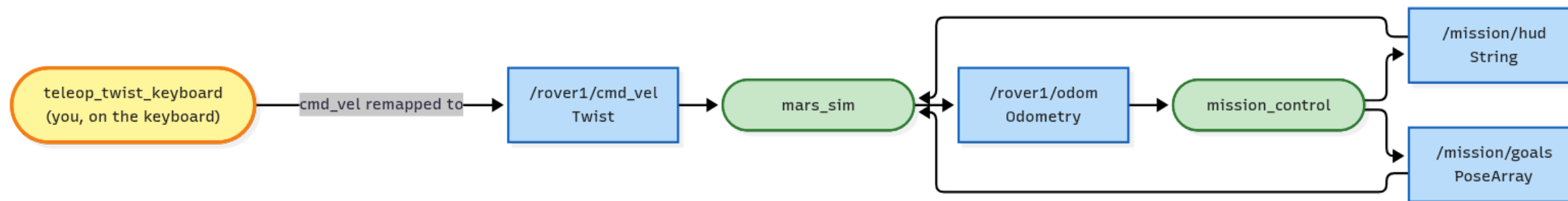
Goal: drive rover1 off the lander to beacon 1

Stars: ≤ 60 s ★★ ★ · ≤ 2.5 min ★★

► 0 1 2 3 4 5 6 7 8 9 10 11 12



Three nodes, no wires between them



- Teleop turns your key presses into `Twist` messages; `-r cmd_vel:=/rover1/cmd_vel` points them at rover1
- The simulator moves the rover and publishes where it is on `/rover1/odom`
- Mission control reads the odometry, and sends the objectives (`/mission/hud`) and the beacon (`/mission/goals`) back

Run it

```
# terminal 1
cd ~/mars_rover
colcon build --symlink-install
source install/setup.bash
ros2 launch mission_control mission.launch.py mission:=0
```

```
# terminal 2
source ~/mars_rover/install/setup.bash
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -r cmd_vel:=/rover1/cmd_vel
```

Click into terminal 2: **i** forward, **j** / **l** turn, **k** stop. Drive to beacon 1.

Can you...

- ☐ build the workspace with `colcon build`?
- ☐ explain why every new terminal needs `source`?
- ☐ launch a mission and say what each part of the window shows?
- ☐ drive the rover with teleop and explain what `-r cmd_vel:=/rover1/cmd_vel` does?
- ☐ name the three nodes that were running?

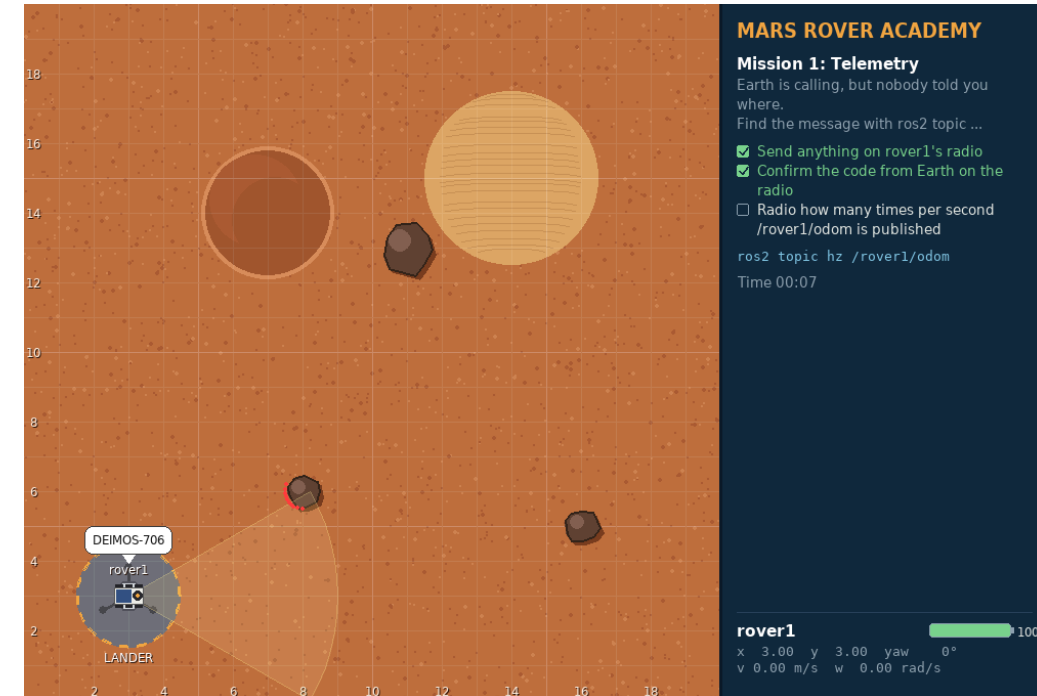
Mission 1: Telemetry

You'll learn: nodes, topics and messages, and the `ros2 node` / `ros2 topic` / `ros2 interface` tools

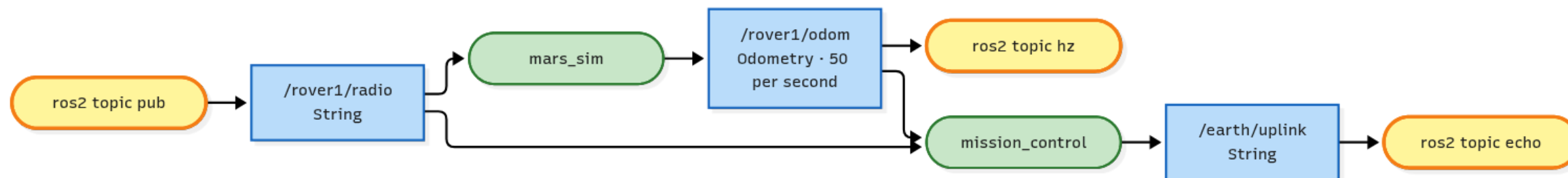
Goal: send something on the radio, confirm the code Earth sends you, and radio how often `/rover1/odom` is published

Stars: ≤ 3 min ★★ ★ · ≤ 7 min ★★

✓0 ▶ 1 2 3 4 5 6 7 8 9 10 11 12



You listen in, then you talk



- `ros2 topic echo` and `ros2 topic hz` are listeners, just like mission control
- `ros2 topic pub` is a publisher: anything you post on `/rover1/radio` shows up above the rover
- The message from Earth is just another topic someone is publishing

Run it

```
ros2 launch mission_control mission.launch.py mission:=1      # terminal 1
```

```
ros2 node list          # terminal 2: who is running?
ros2 topic list         # which topics exist?
ros2 topic echo --once /rover1/odom    # read one message
ros2 topic info /rover1/radio          # its type
ros2 interface show std_msgs/msg/String # the fields of that type
ros2 topic pub --once /rover1/radio std_msgs/msg/String "{data: 'Hello, Earth!'}"
ros2 topic hz /rover1/odom             # how often? (Ctrl+C)
```

Your turn: find the topic with the message from Earth and confirm its code.

Can you...

- ☐ list the nodes and topics in a running system?
- ☐ listen to a topic with `ros2 topic echo`?
- ☐ find a topic's message type and the fields of that type?
- ☐ send a message yourself with `ros2 topic pub`?
- ☐ measure how often a topic is published?
- ☐ explain why a publisher doesn't need to know who is listening?

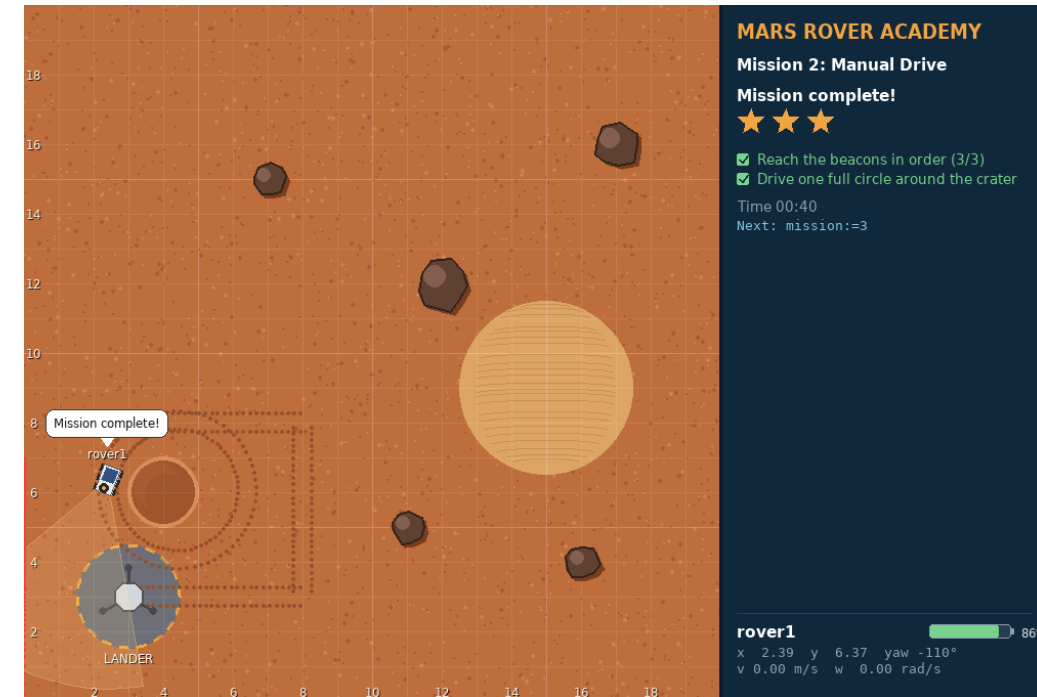
Mission 2: Manual Drive

You'll learn: `geometry_msgs/msg/Twist`: `linear.x` is forward speed (m/s), `angular.z` is turn rate (rad/s). Radians. The 1-second rule, top speeds.

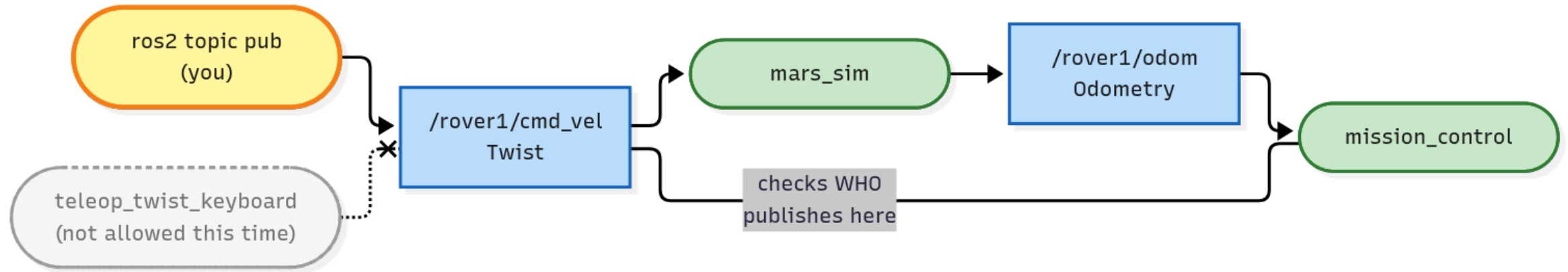
Goal: reach beacons 1, 2, 3 in order and drive one full circle around the crater, by publishing commands yourself (no teleop)

Stars: ≤ 4 min ★★ ★ · ≤ 8 min ★★

✓0 ✓1 ▶2 3 4 5 6 7 8 9 10 11 12



You become the steering wheel



- This time **you** publish on `/rover1/cmd_vel`, with `ros2 topic pub`
- Teleop is not allowed: mission control checks *who* publishes on that topic

Run it

```
ros2 launch mission_control mission.launch.py mission:=2      # terminal 1
```

```
T="/rover1/cmd_vel geometry_msgs/msg/Twist"
ros2 topic pub --rate 1 --times 5 $T "{linear: {x: 1.0}}"      # 5 m east to beacon 1
ros2 topic pub --once $T "{angular: {z: 1.5708}}"              # turn left 90° (now facing north)
ros2 topic pub --rate 1 --times 5 $T "{linear: {x: 1.0}}"      # to beacon 2
ros2 topic pub --once $T "{angular: {z: 1.5708}}"              # now facing west
ros2 topic pub --rate 1 --times 4 $T "{linear: {x: 1.0}}"      # to beacon 3
ros2 topic pub --rate 1 $T "{linear: {x: 1.0}, angular: {z: 0.5}}" # circle, then Ctrl+C
```

Options go right after `pub`. Wait for the rover to stop between commands.

Can you...

- ☐ say what `linear.x` and `angular.z` in a `Twist` mean, with their units?
- ☐ convert an angle in degrees to radians?
- ☐ work out how far a command moves the rover, using the 1-second rule and the top speed?
- ☐ use `--once`, `--rate` and `--times`, and put them in the right place?
- ☐ drive a circle and predict its radius?
- ☐ explain how mission control can tell teleop from your commands?

Mission 3: Mission Control

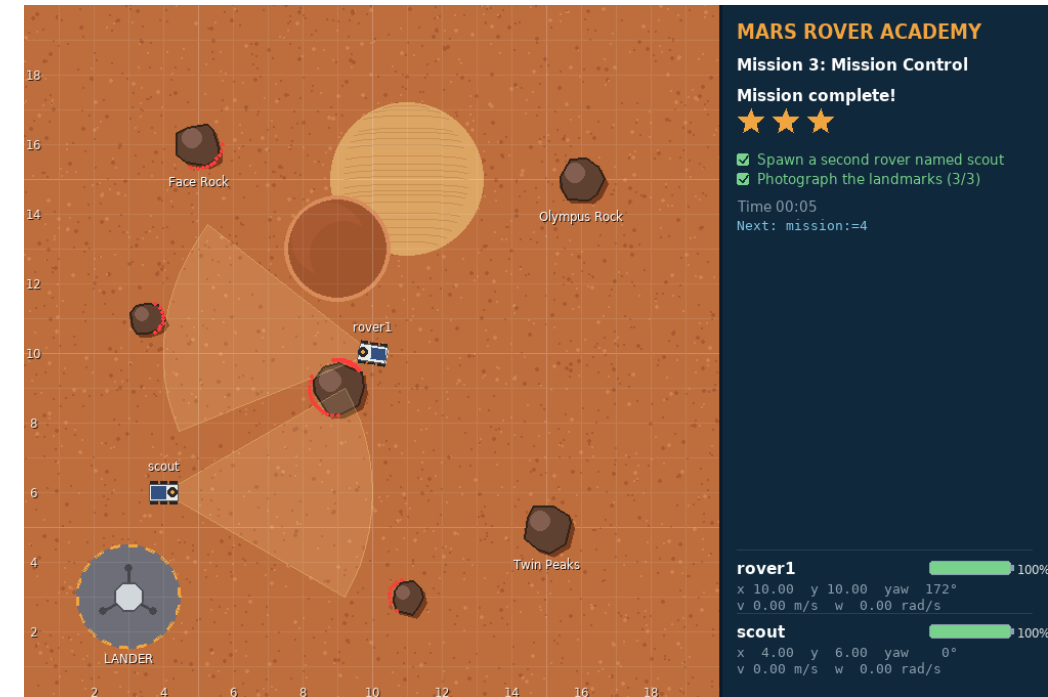
You'll learn: services: one request, one response.

`ros2 service list / type / call`, reading a `.srv` definition, `std_srvs/Trigger` (success + message)

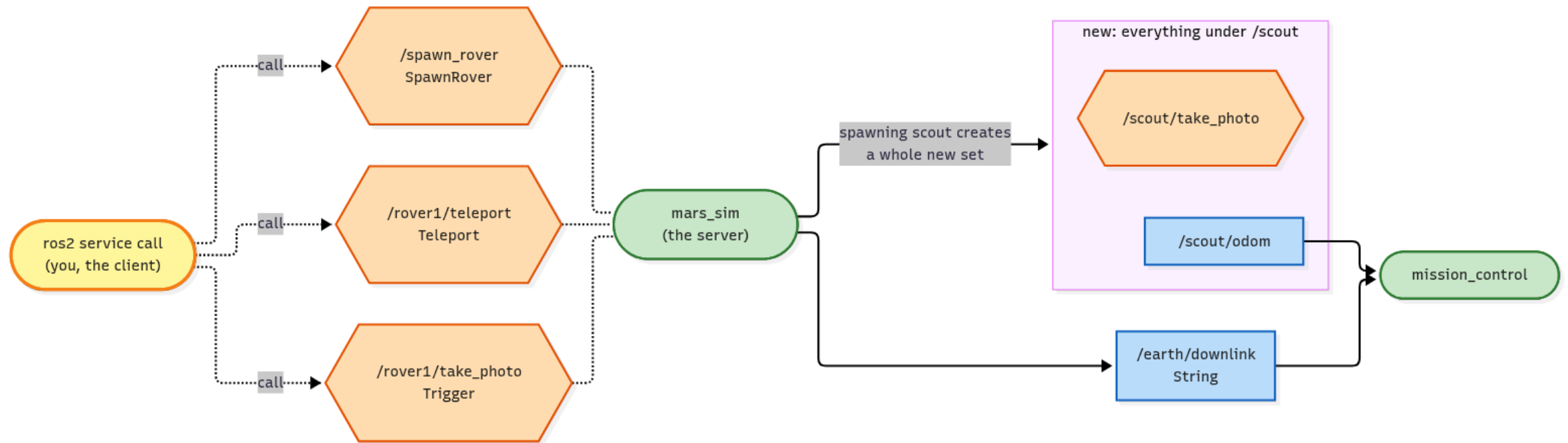
Goal: spawn a second rover named `scout`, and photograph Olympus Rock (16, 15), Twin Peaks (15, 5) and Face Rock (5, 16)

Stars: ≤ 5 min ★★ ★ · ≤ 10 min ★★

✓0 ✓1 ✓2 ▶3 4 5 6 7 8 9 10 11 12



Ask and get an answer



- A service has a **client** (you) and a **server** (the simulator)
- Every call is one request and one response: `success` says whether it worked, `message` says why
- Spawning `scout` creates a whole new set of topics and services under `/scout`

Run it

```
ros2 launch mission_control mission.launch.py mission:=3      # terminal 1
```

```
ros2 service list
ros2 interface show mars_interfaces/srv/SpawnRover      # above --- the request, below the response
ros2 service call /spawn_rover mars_interfaces/srv/SpawnRover "{name: scout, x: 4.0, y: 6.0, yaw: 0.0}"

ros2 service call /rover1/teleport mars_interfaces/srv/Teleport "{x: 14.0, y: 15.0, yaw: 0.0}"
ros2 service call /rover1/take_photo std_srvs/srv/Trigger
# the same for Twin Peaks (13, 5) and Face Rock (3, 16): 2 m west of it, facing east
```

Read each response: a failed photo tells you what is wrong.

Can you...

- ☐ list the services and find the type a service uses?
- ☐ read a service definition and say which part is the request and which the response?
- ☐ call a service from the terminal and read `success` and `message` in a `Trigger` response?
- ☐ explain why spawning a rover creates a whole new set of topics and services?
- ☐ say when to use a topic and when to use a service?

Mission 4: Survey Square

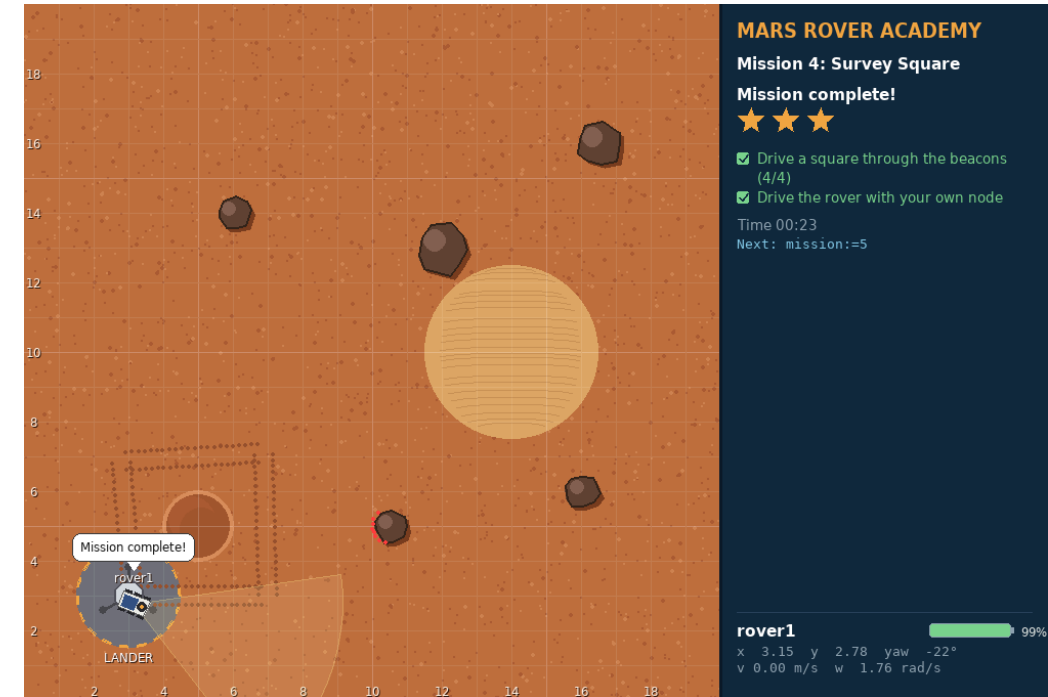
You'll learn: creating a Python package, a node with a **publisher** and a **timer**, registering it in

`setup.py`

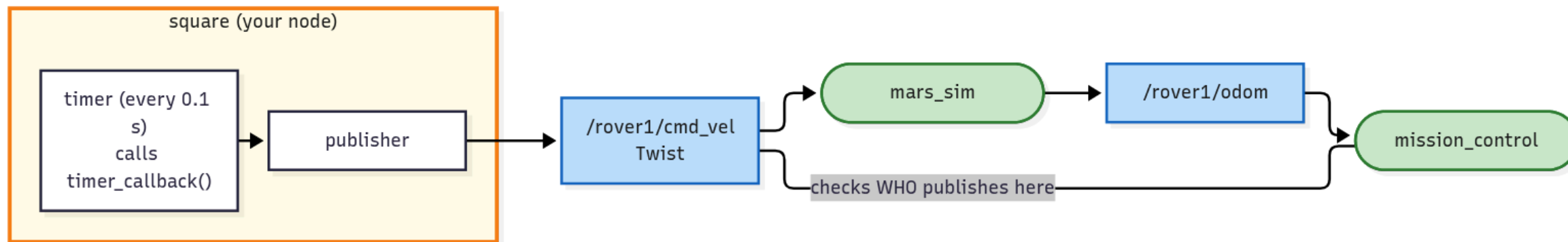
Goal: drive a 4 m square through 4 beacons with your own node

Stars: ≤ 30 s ★★ ★ · ≤ 45 s ★★ · the clock starts when the rover moves

✓0 ✓1 ✓2 ✓3 ▶4 5 6 7 8 9 10 11 12



Your first node in the graph



- Inside `square`: a **timer** fires every 0.1 s and the callback **publishes** a `Twist`
- From the outside it looks exactly like teleop: one more publisher on `/rover1/cmd_vel`
- It never reads `/rover1/odom`, so it can't notice when it goes wrong (open loop)

Run it

```
cd ~/mars_rover/src
ros2 pkg create --build-type ament_python my_rover --license Apache-2.0 \
  --dependencies rclpy geometry_msgs nav_msgs sensor_msgs std_msgs std_srvs mars_interfaces
```

Write `src/my_rover/my_rover/square.py`, then register it in `setup.py`:

```
'square = my_rover.square:main',
```

```
cd ~/mars_rover
colcon build --symlink-install --packages-select my_rover
source install/setup.bash
ros2 launch mission_control mission.launch.py mission:=4      # terminal 1
ros2 run my_rover square                                     # terminal 2
```

A publisher and a timer

```
class Square(Node):
    def __init__(self):
        super().__init__('square') # node name (shows up in ros2 node list)
        # publisher: sends Twist to /rover1/cmd_vel (10 = message queue size)
        self.publisher = self.create_publisher(Twist, '/rover1/cmd_vel', 10)
        # timer: call timer_callback every 0.1 s
        self.timer = self.create_timer(0.1, self.timer_callback)

    def timer_callback(self):
        msg = Twist()
        msg.linear.x = 1.0 # 1 m/s forward
        msg.angular.z = 0.5 # 0.5 rad/s to the left
        self.publisher.publish(msg)
```

`main()` is always: `rclpy.init()`, create the node, `rclpy.spin(node)`, shut down.

Can you...

- ☐ create a Python package with `ros2 pkg create` ?
- ☐ write a node with a publisher and a timer?
- ☐ register a program in `setup.py` and start it with `ros2 run` ?
- ☐ say when you have to build and source again, and when you don't?
- ☐ explain the difference between a package, an executable and a node?
- ☐ explain why this node is "open loop", and why it ends a little off its start?

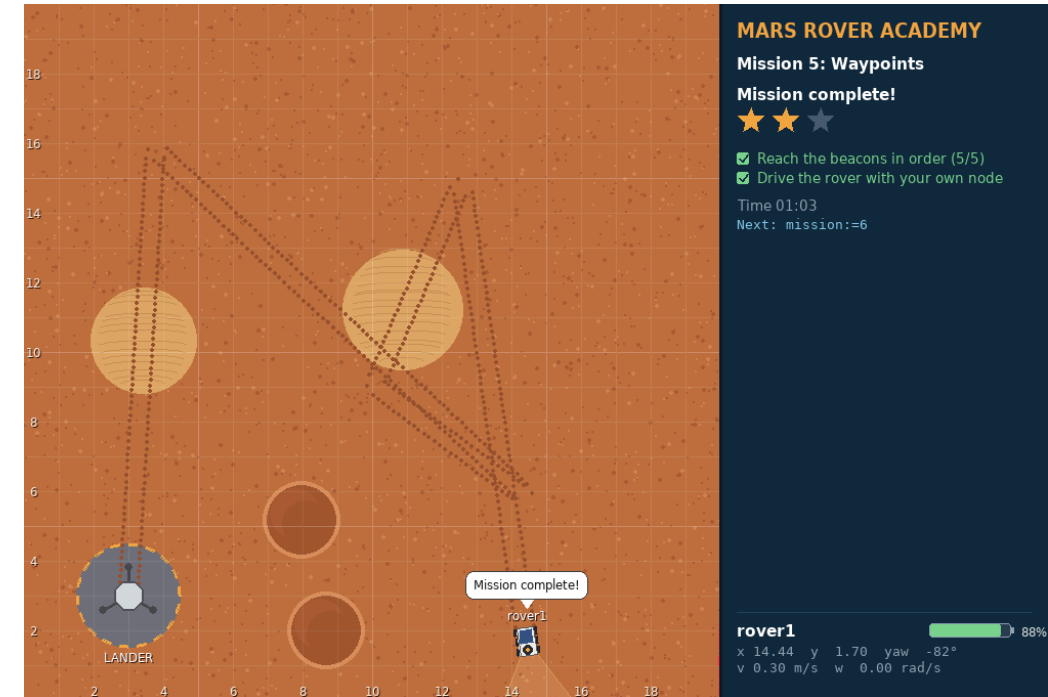
Mission 5: Waypoints

You'll learn: subscribers and callbacks, `nav_msgs/Odometry` and quaternions, closed-loop control, `atan2`, a P controller

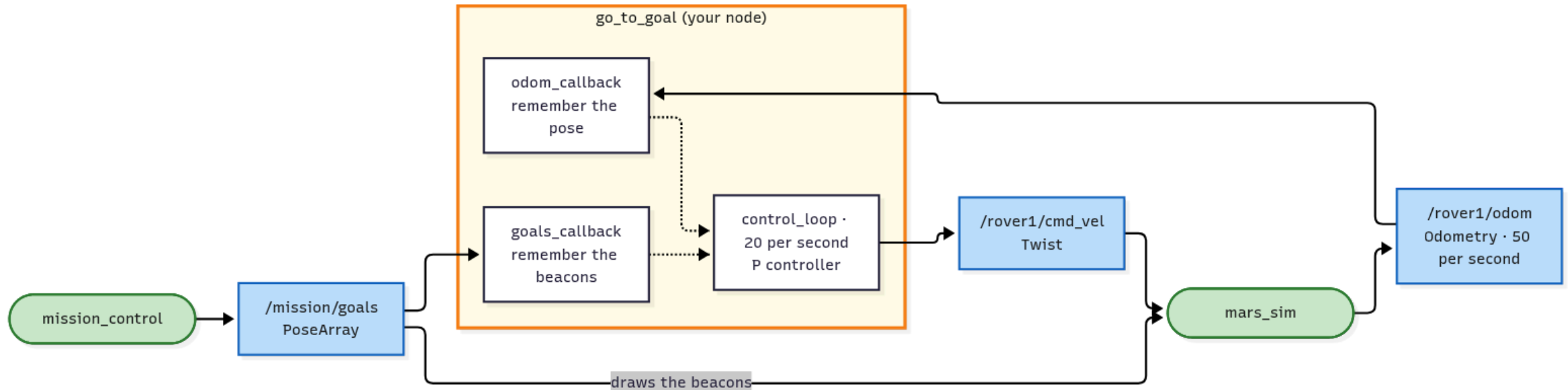
Goal: reach 5 beacons at random positions, in order, with your own node; sand makes the rover slip

Stars: ★★ ★ ≤ 1.3 s per metre of route + 8 s

✓0 ✓1 ✓2 ✓3 ✓4 ▶5 6 7 8 9 10 11 12



Close the loop



- Two **subscribers**: the odometry and the beacons. Their callbacks only remember the latest message
- A timer runs `control_loop` 20 times a second: compare where you are with where you want to be
- The arrow from the odometry back into your node is what makes it closed loop

Run it

```
ros2 topic echo --once /mission/goals      # where are the beacons?  
ros2 topic echo --once /rover1/odom       # where am I? (orientation is a quaternion)
```

Write `src/my_rover/my_rover/go_to_goal.py`, add to `setup.py`:

```
'go_to_goal = my_rover.go_to_goal:main',
```

```
cd ~/mars_rover  
colcon build --symlink-install --packages-select my_rover  
source install/setup.bash  
ros2 launch mission_control mission.launch.py mission:=5      # terminal 1  
ros2 run my_rover go_to_goal                                   # terminal 2
```

Callbacks remember, the timer decides

```
def yaw_from_quaternion(q) -> float:
    """The rover only turns about z, so this is all of the quaternion we need."""
    return math.atan2(2.0 * (q.w * q.z + q.x * q.y), 1.0 - 2.0 * (q.y * q.y + q.z * q.z))

def control_loop(self):
    cmd = Twist() # start from zero velocity
    if self.pose is not None and self.goals:
        x, y, yaw = self.pose
        goal_x, goal_y = self.goals[0]
        dx = goal_x - x
        dy = goal_y - y
        distance = math.hypot(dx, dy)
        # angle we should face minus angle we face = heading error (wrapped to -pi..pi)
        error = math.atan2(dy, dx) - yaw
        error = math.atan2(math.sin(error), math.cos(error))

        cmd.angular.z = K_ANGULAR * error
        if abs(error) < 0.5: # only drive once we roughly face the beacon
            cmd.linear.x = min(K_LINEAR * distance, MAX_SPEED)
    self.publisher.publish(cmd)
```

Can you...

- ☐ write a subscriber whose callback only stores the data?
- ☐ explain why the decisions happen in a timer and not in the callback?
- ☐ get x, y and the yaw out of a `nav_msgs/Odometry` message?
- ☐ compute the distance and heading error to a point with `hypot` and `atan2`, wrapped into $-\pi.. \pi$?
- ☐ write a P controller and say what each gain changes?
- ☐ explain why sand ruins an open-loop plan but not a closed loop?

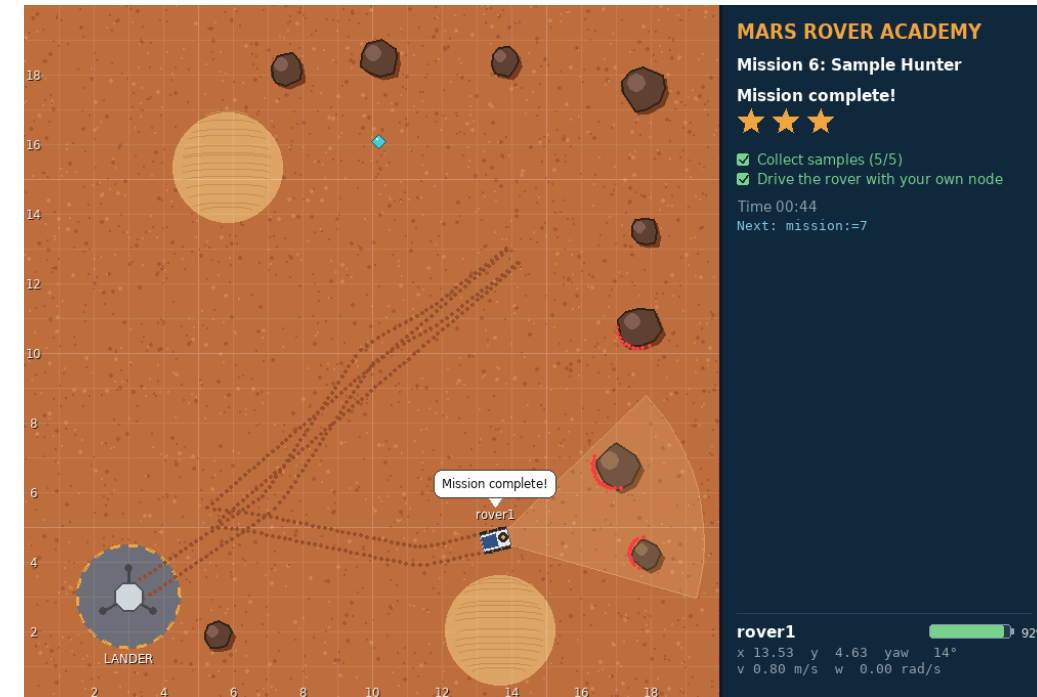
Mission 6: Sample Hunter

You'll learn: reading a camera (range and bearing) and a laser (`LaserScan`), a **service client** in code with `call_async`

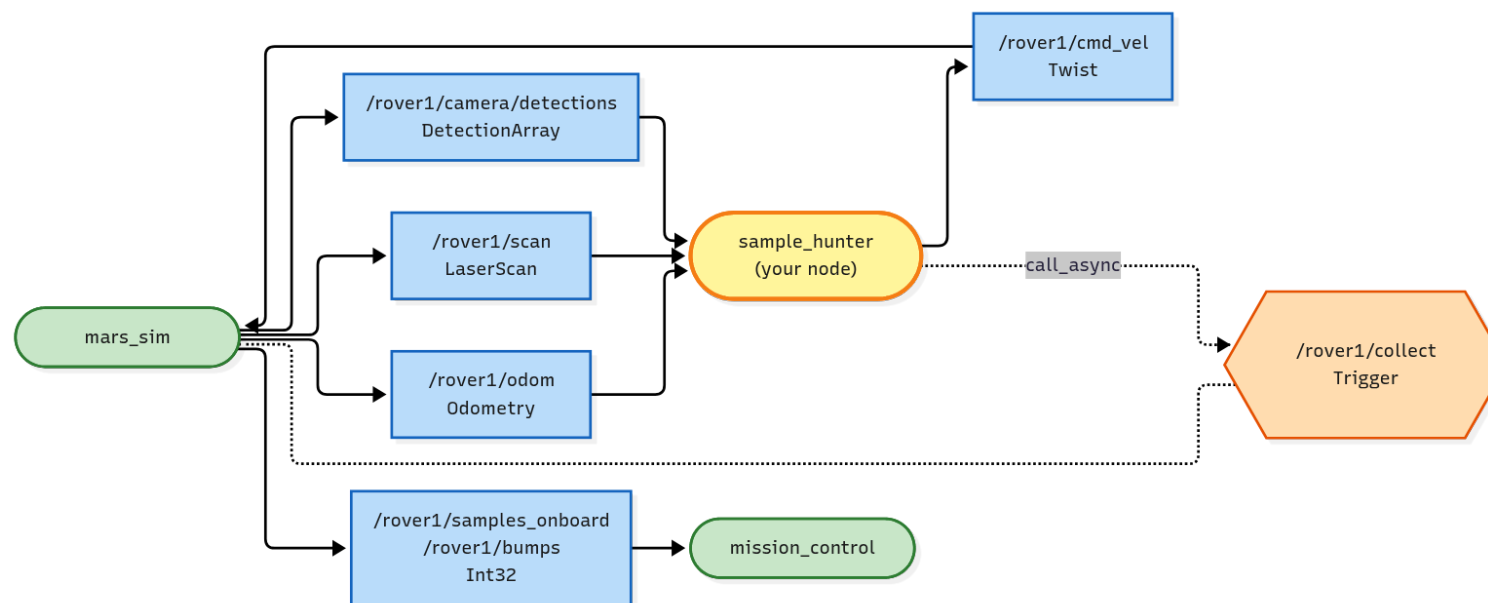
Goal: find and collect 5 samples with your own node, without bumping into rocks

Stars: ≤ 90 s ★★ ★ · ≤ 180 s ★★ · a bump: ★
★ at most

✓0 ✓1 ✓2 ✓3 ✓4 ✓5 ▶6 7 8 9 10 11 12



Two sensors and a service in code



- `/rover1/camera/detections` says what the rover sees: kind, range and bearing
- `/rover1/scan` says how far the nearest thing is in every direction
- Your node calls `/rover1/collect` itself, with `call_async`, without stopping to wait

Run it

```
ros2 interface show mars_interfaces/msg/Detection
ros2 topic echo --once /rover1/scan --no-arr
```

Write `src/my_rover/my_rover/sample_hunter.py`, add to `setup.py`:

```
'sample_hunter = my_rover.sample_hunter:main',
```

```
cd ~/mars_rover
colcon build --symlink-install --packages-select my_rover
source install/setup.bash
ros2 launch mission_control mission.launch.py mission:=6      # terminal 1
ros2 run my_rover sample_hunter                               # terminal 2
```

Sample in sight: chase and collect. Nothing: patrol. Rock ahead: steer around it.

Calling a service without waiting

```
self.collect_client = self.create_client(Trip, '/rover1/collect')

def collect(self):
    # only send a new request once the previous one was answered, so we don't spam
    if self.collect_future is None or self.collect_future.done():
        self.collect_future = self.collect_client.call_async(Trip.Request())

# in control_loop:
samples = [d for d in self.detections if d.kind == 'sample']
if samples:
    target = min(samples, key=lambda d: d.range) # the closest one
    cmd.angular.z = K_ANGULAR * target.bearing # bearing is already the heading error
    if abs(target.bearing) < 0.5:
        cmd.linear.x = min(0.8 * target.range, MAX_SPEED)
    if target.range < COLLECT_RANGE and abs(target.bearing) < 0.4:
        self.collect()
```

Never wait inside a callback: `spin()` runs one callback at a time, so the answer could never arrive (deadlock).

Can you...

- ☐ pick the closest sample from a `DetectionArray`, and say what range and bearing mean?
- ☐ find the angle of laser ray `i` and the closest obstacle in front?
- ☐ create a service client and call it with `call_async`?
- ☐ say what a future is and when to check `.done()`?
- ☐ explain why waiting inside a callback causes a deadlock?
- ☐ describe the hunter's decision: chase, patrol, or steer around a rock?

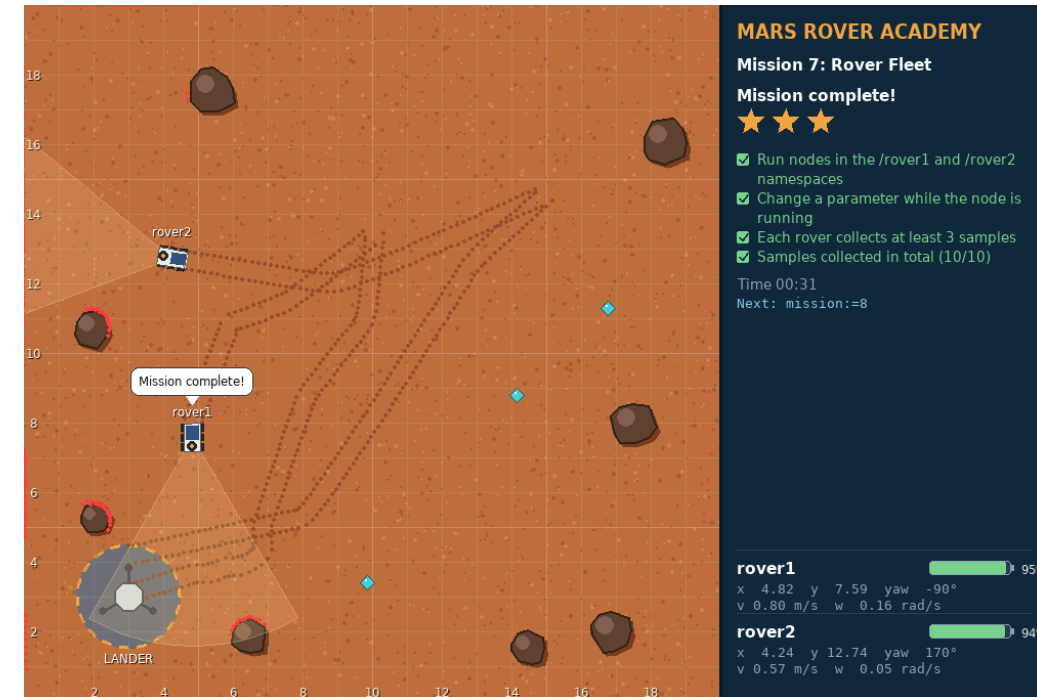
Mission 7: Rover Fleet

You'll learn: **namespaces** (same code, other robot), **parameters** (settings you change from outside), **launch files** (the whole fleet in one command)

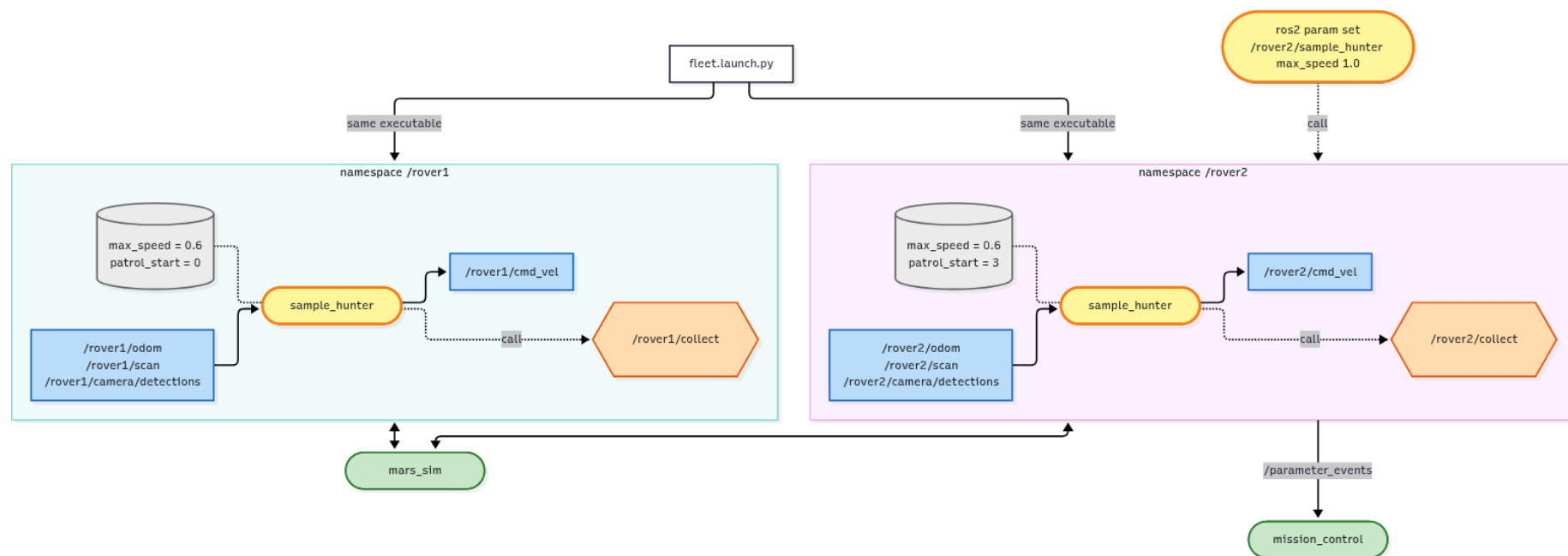
Goal: two rovers run the same hunter node, you change a parameter live, 10 samples in total

Stars: $\leq 90\text{ s}$ ★★ ★ $\cdot \leq 180\text{ s}$ ★★

✓0 ✓1 ✓2 ✓3 ✓4 ✓5 ✓6 ▶7 8 9 10 11 12



One program, two robots



- The same `sample_hunter` runs twice, once in `/rover1` and once in `/rover2`
- Relative names (`odom` , `cmd_vel`) land inside the node's namespace
- **Parameters** (grey) give each copy its own settings; `ros2 param set` changes them live

Run it

```
ros2 launch mission_control mission.launch.py mission:=7           # terminal 1
ros2 run my_rover sample_hunter --ros-args -r __ns:=/rover1        # terminal 2
ros2 run my_rover sample_hunter --ros-args -r __ns:=/rover2 -p patrol_start:=3  # terminal 3
```

```
ros2 node list              # /rover1/sample_hunter, /rover2/sample_hunter
ros2 param list /rover2/sample_hunter
ros2 param set /rover2/sample_hunter max_speed 1.0  # rover2 speeds up right away
```

Or the whole fleet with one command, after writing `launch/fleet.launch.py`:

```
ros2 launch my_rover fleet.launch.py
```

Relative names, parameters, launch

```
self.declare_parameter('max_speed', 0.6)
self.declare_parameter('patrol_start', 0)    # which patrol point to start from
# relative names (no leading /) -> they live under the node's namespace
self.create_subscription(Odometry, 'odom', self.odom_callback, 10)
self.publisher = self.create_publisher(Twist, 'cmd_vel', 10)
self.collect_client = self.create_client(Trigger, 'collect')
```

```
Node(
    package='my_rover',
    executable='sample_hunter',
    namespace='rover2',
    parameters=[{'patrol_start': 3}],
),
```

Can you...

- ☐ run the same node for two rovers with `__ns` ?
- ☐ explain how a relative name like `'cmd_vel'` becomes `/rover2/cmd_vel` ?
- ☐ declare a parameter in code and read it?
- ☐ set a parameter at start-up and change it while the node runs?
- ☐ write a launch file and install it from `setup.py` ?

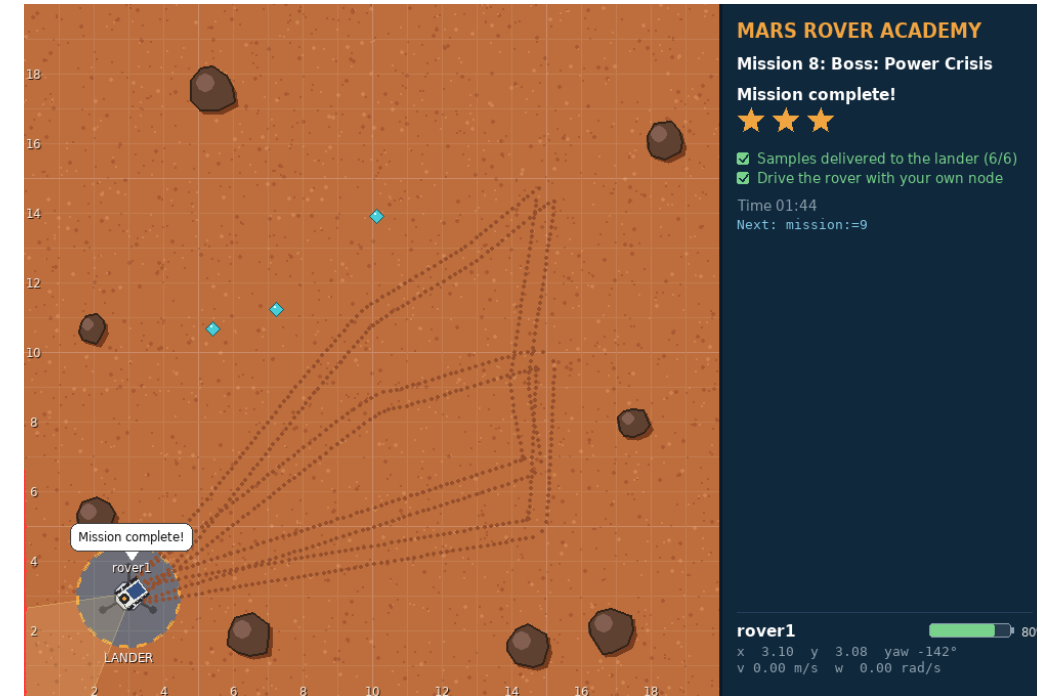
Mission 8 (boss): Power Crisis

You'll learn: putting topics and services together, a battery to manage, and thinking in **states**

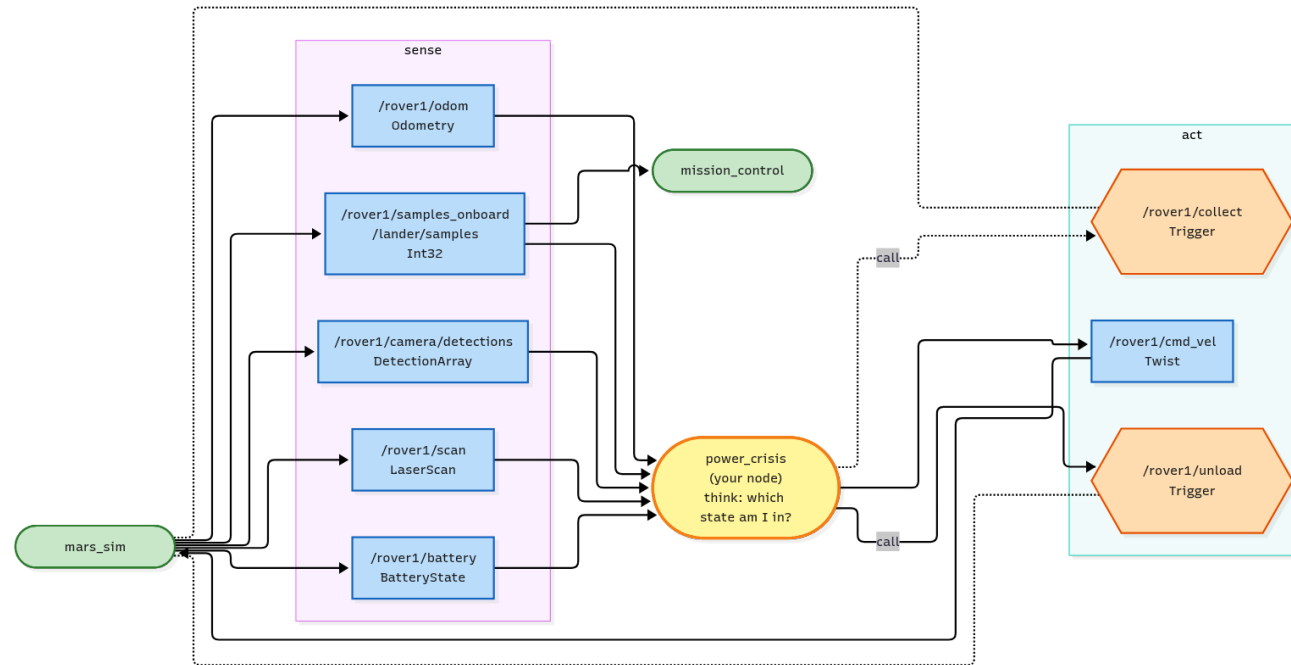
Goal: bring 6 samples to the lander, starting with 35 % battery that drains 4 times faster; run flat and the mission fails

Stars: ≤ 170 s ★★ ★ · ≤ 340 s ★★

✓0 ✓1 ✓2 ✓3 ✓4 ✓5 ✓6 ✓7 ►8 9 10 11 12



Sense, think, act



- **Sense:** odometry, camera, laser, battery, samples on board, samples at the lander
- **Think:** explore, return or charge?
- **Act:** drive on `cmd_vel`, call `collect` and `unload`

Run it

In	Out
/rover1/odom , /rover1/camera/detections , /rover1/scan	/rover1/cmd_vel
/rover1/battery , /rover1/samples_onboard , /lander/samples	services /rover1/collect , /rover1/unload

Copy the skeleton from the guide to `src/my_rover/my_rover/power_crisis.py` , add `'power_crisis = my_rover.power_crisis:main'` , to `setup.py` , then:

```
cd ~/mars_rover
colcon build --symlink-install --packages-select my_rover
source install/setup.bash
ros2 launch mission_control mission.launch.py mission:=8      # terminal 1
ros2 run my_rover power_crisis                                # terminal 2
```

Think in states

```
def control_loop(self):
    if self.pose is None or self.battery is None:
        return
    needed = COST_PER_METRE * self.distance_home() + RESERVE    # battery to get home from here
    if self.state == 'explore':
        # TODO 4: switch to 'return' when the battery is below `needed`,
        #           or when delivered + on board is already enough for the lander
        cmd = self.explore()
    elif self.state == 'return':
        # TODO 5: steer to the lander; closer than 0.4 m: stop and switch to 'charge'
        cmd = Twist()
    else: # 'charge'
        cmd = Twist()    # stand still: the rover only charges while it is parked
        # TODO 6: unload while anything is on board; once the battery is FULL
        #           and the lander still needs samples, go back to 'explore'
    self.avoid_obstacles(cmd)
    self.publisher.publish(cmd)
```

Can you...

- ☐ list a robot node's inputs and outputs as sense, think, act?
- ☐ describe a multi-step job as states, and what makes the rover change state?
- ☐ read the battery from `sensor_msgs/BatteryState` and decide when to turn back?
- ☐ call two different services from one node without spamming them?
- ☐ turn a skeleton into a working node, one TODO at a time?

Part 2: the arms

Missions 9 to 12: joints, frames, the drill and pick-and-place. Mission launches switch the arms on automatically.

Mission 9: Arm Check

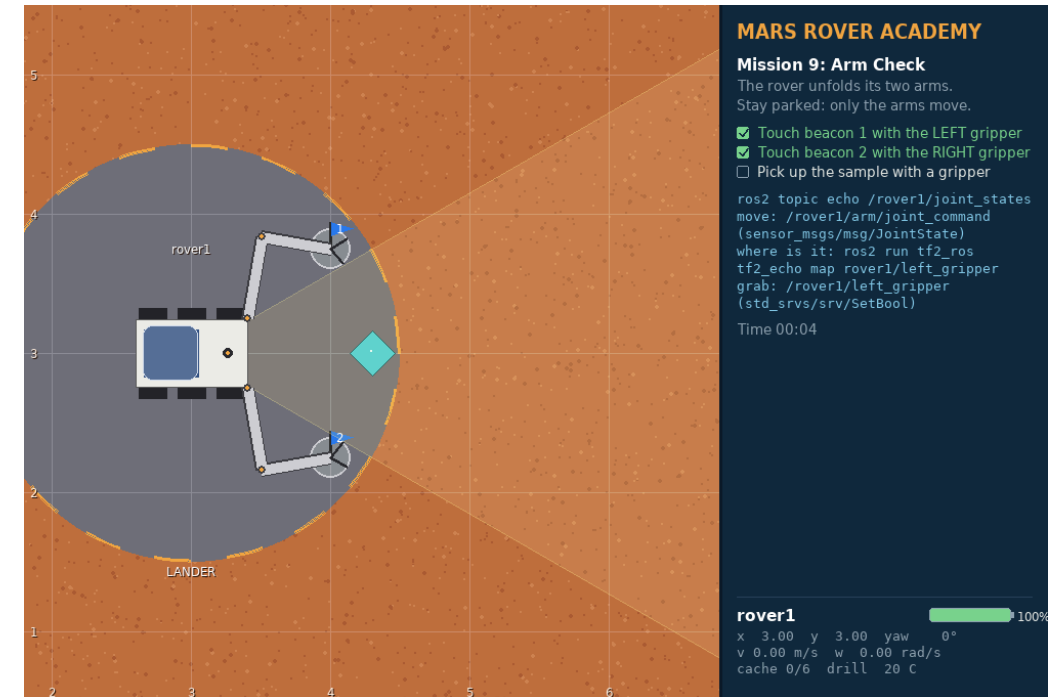
You'll learn: joint control with

`sensor_msgs/JointState`, grippers as
`std_srvs/SetBool` services, forward kinematics,
`tf2_echo`

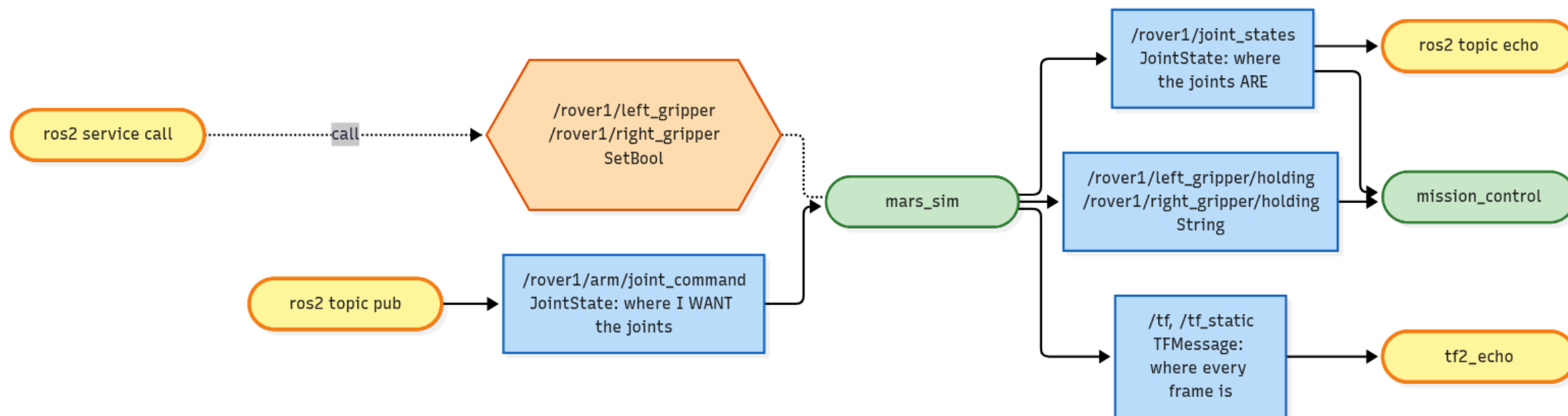
Goal: touch beacon 1 with the left gripper, beacon 2 with the right, then pick up the sample. The rover stays parked.

Stars: ≤ 4 min ★★ ★ · ≤ 8 min ★★

✓0 ✓1 ✓2 ✓3 ✓4 ✓5 ✓6 ✓7 ✓8 ▶9 10 11 12



Arms are just more topics and services



- `arm/joint_command` is where you **want** the joints, `joint_states` is where they **are**
- tf2 knows where every part of the arm is: `tf2_echo map rover1/left_gripper`
- Each gripper is a `SetBool` service: `true` closes it, `false` opens it

Run it

```
ros2 launch mission_control mission.launch.py mission:=9           # terminal 1
```

```
ros2 topic echo --once /rover1/joint_states                        # where the joints are
ros2 run tf2_ros tf2_echo map rover1/left_gripper                # where the gripper is
ros2 topic pub --once /rover1/arm/joint_command sensor_msgs/msg/JointState \
    "{name: [left_shoulder, left_elbow], position: [1.39, -1.57]}"
ros2 topic pub --once /rover1/arm/joint_command sensor_msgs/msg/JointState \
    "{name: [left_shoulder, left_elbow], position: [0.23, -1.12]}"
ros2 service call /rover1/left_gripper std_srvs/srv/SetBool "{data: true}"
```

Guess angles, check the gripper with `tf2_echo`, adjust.

Can you...

- ☐ read `/rover1/joint_states` and say which angle belongs to which joint?
- ☐ move one arm with `arm/joint_command` from the terminal?
- ☐ find where a gripper is with `ros2 run tf2_ros tf2_echo`?
- ☐ close and open a gripper with a `SetBool` service and read its answer?
- ☐ work out where the gripper is from two joint angles (forward kinematics)?

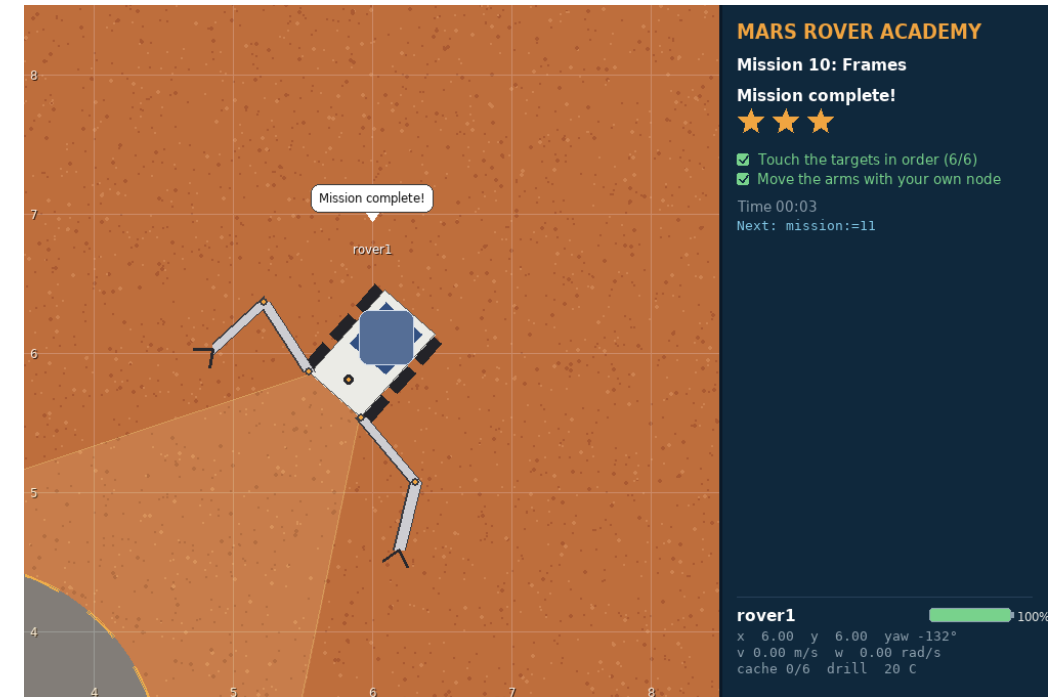
Mission 10: Frames

You'll learn: coordinate frames with **tf2** (`Buffer` , `TransformListener` , `transform()`), forward and **inverse kinematics** of a 2-link arm

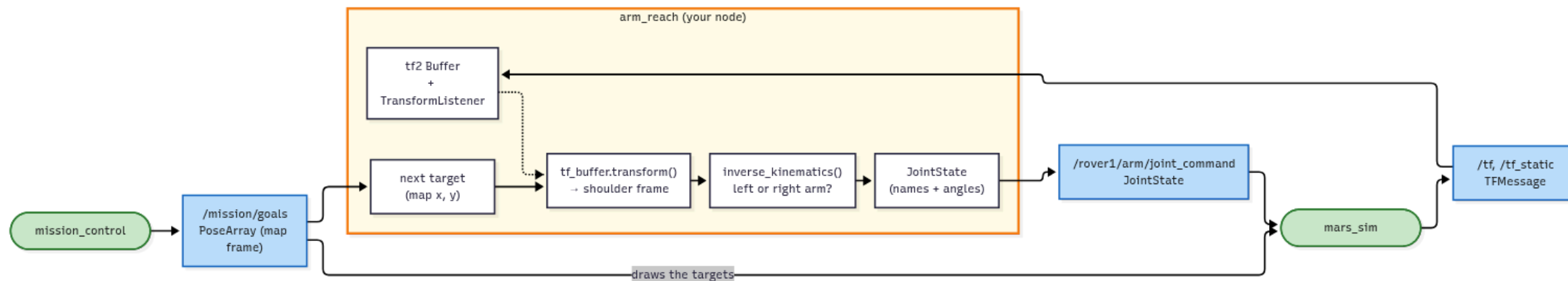
Goal: the rover is parked at a random heading: touch 6 targets in order by computing the joint angles in your own node

Stars: $\leq 8\text{ s}$ ★★ ★ · $\leq 20\text{ s}$ ★★

✓0 ✓1 ✓2 ✓3 ✓4 ✓5 ✓6 ✓7 ✓8 ✓9 ▶10 11 12



From a map point to joint angles



- The targets come in the **map** frame on `/mission/goals`
- tf2 turns each one into the **shoulder** frame: no frame maths in your code
- **Inverse kinematics** turns that point into two joint angles on `/rover1/arm/joint_command`

Run it

Add `tf2_ros`, `tf2_geometry_msgs` and `action_msgs` to `package.xml`, write `arm_kinematics.py` and test it on its own:

```
cd ~/mars_rover/src/my_rover/my_rover
python3 -c "from arm_kinematics import *; print(forward_kinematics(*inverse_kinematics(0.5, 0.3, 'left')))"
```

Then `arm_reach.py`, add `'arm_reach = my_rover.arm_reach:main'`, to `setup.py`, and:

```
cd ~/mars_rover
colcon build --symlink-install --packages-select my_rover
source install/setup.bash
ros2 launch mission_control mission.launch.py mission:=10           # terminal 1
ros2 run my_rover arm_reach                                         # terminal 2
```

tf2, then inverse kinematics

```
self.tf_buffer = Buffer()                # remembers the frames for a few seconds
self.tf_listener = TransformListener(self.tf_buffer, self) # fills it from /tf and /tf_static

point = PointStamped()
point.header.frame_id = 'map'            # the frame the numbers are in
point.point.x, point.point.y = 5.837, 4.913
p = self.tf_buffer.transform(point, 'rover1/left_shoulder').point # p.x = 0.52, p.y = 0.35
```

```
def inverse_kinematics(x: float, y: float, side: str):
    """Joint angles (q1, q2) that put the gripper at (x, y) in the shoulder frame, or None."""
    cos_q2 = (x * x + y * y - L1 * L1 - L2 * L2) / (2 * L1 * L2) # law of cosines
    if abs(cos_q2) > 1.0 + 1e-9:
        return None # out of reach
    q2 = math.acos(max(-1.0, min(1.0, cos_q2))) # rounding can give 1.0000000000000002
    if side == 'left':
        q2 = -q2 # bend the elbow outwards, away from the body
    q1 = math.atan2(y, x) - math.atan2(L2 * math.sin(q2), L1 + L2 * math.cos(q2))
    q1 = math.atan2(math.sin(q1), math.cos(q1)) # keep it in -pi..pi
    return q1, q2
```

Can you...

- ☐ convert a point from the map frame into the rover frame by hand?
- ☐ do the same with a tf2 `Buffer`, `TransformListener` and `transform()`?
- ☐ explain forward versus inverse kinematics?
- ☐ compute both joint angles of a 2-link arm with the law of cosines, and say when a point is out of reach?
- ☐ test your maths without starting ROS?

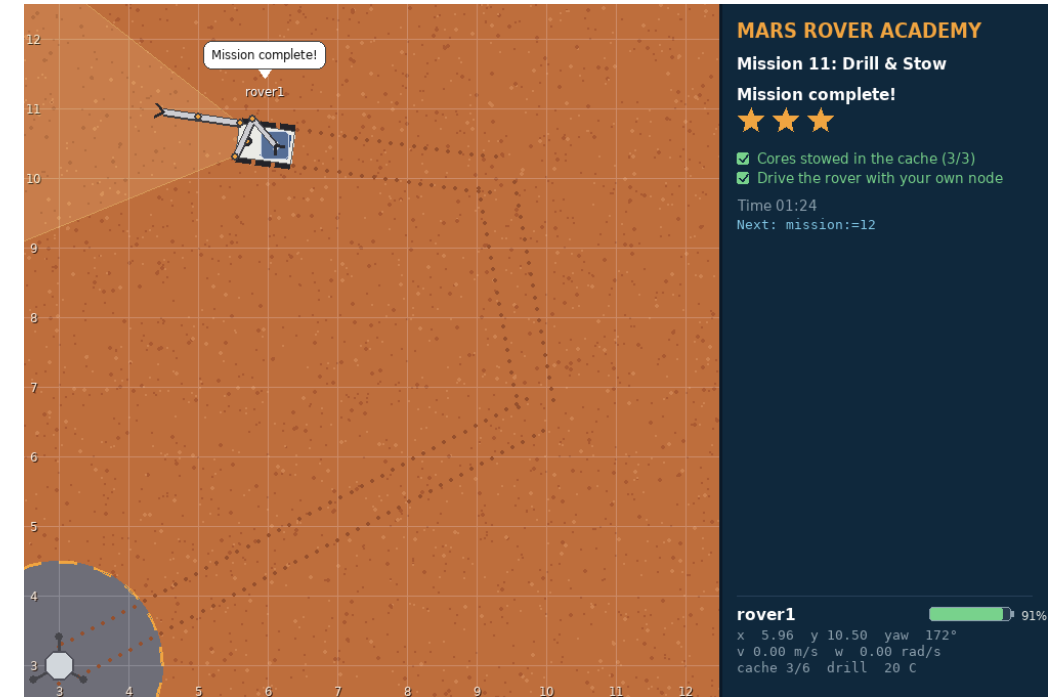
Mission 11: Drill & Stow

You'll learn: **actions** in code: goal, feedback, cancel, result. Pick and place as a state machine.

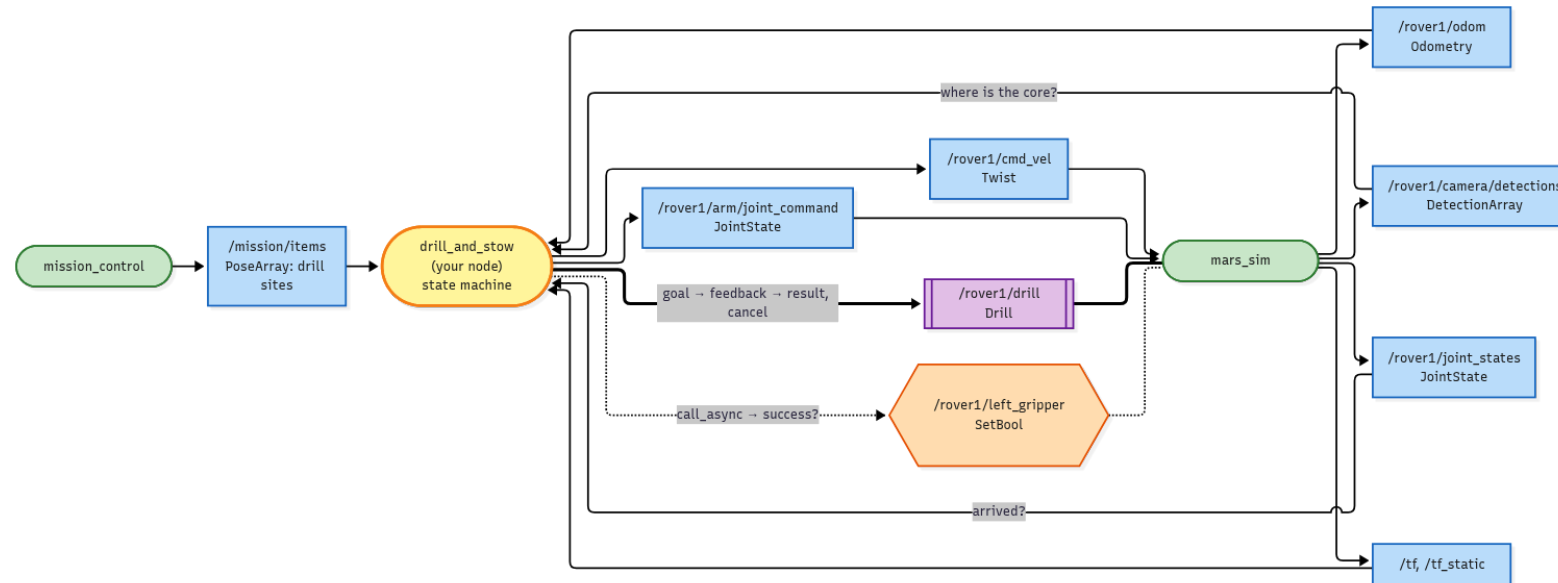
Goal: drill 3 sites, stopping before the bit overheats (80 °C breaks it and ends the mission), and stow each core in the cache

Stars: ≤ 2 min ★★ ★ · ≤ 4 min ★★

✓0 ✓1 ✓2 ✓3 ✓4 ✓5 ✓6 ✓7 ✓8 ✓9 ✓10 ►11 12



A long job you can watch and stop



- The drill is an **action**: one goal, feedback 10 times a second (depth, temperature), one result
- Your node cancels above 70 °C, lets the bit cool and sends the goal again: the hole keeps its depth
- Then the arm drops the core in the cache: tf2 and IK from mission 10

Run it

```
ros2 launch mission_control mission.launch.py mission:=11      # terminal 1
```

```
ros2 interface show mars_interfaces/action/Drill
ros2 action list
ros2 action info /rover1/drill
ros2 action send_goal --feedback /rover1/drill mars_interfaces/action/Drill "{depth: 0.3}"
ros2 topic echo --once /mission/items      # the drill sites
```

Then `drill_and_stow.py`, add it to `setup.py`, build, and `ros2 run my_rover drill_and_stow`.

States: drive → settle → drilling ⇌ cool → reach → grab → stow, then the next site.

An action client

```
self.drill_client = ActionClient(self, Drill, '/rover1/drill')

# 1. send the goal, say which function gets the feedback
future = self.drill_client.send_goal_async(Drill.Goal(depth=0.3), feedback_callback=self.drill_feedback)
future.add_done_callback(self.drill_accepted)

def drill_accepted(self, future):
    self.goal_handle = future.result()           # 2. accepted or rejected?
    if self.goal_handle.accepted:
        self.goal_handle.get_result_async().add_done_callback(self.drill_done)

def drill_feedback(self, msg):                   # 3. ten times a second while it drills
    if msg.feedback.temperature > 70.0:
        self.goal_handle.cancel_goal_async()    # stop it

def drill_done(self, future):                    # 4. the result, whatever happened
    status = future.result().status              # GoalStatus.STATUS_SUCCEEDED / _CANCELED / _ABORTED
    message = future.result().result.message
```

Can you...

- ☐ use `ros2 action list`, `info` and `send_goal --feedback`?
- ☐ send a goal from code and receive feedback and the result?
- ☐ cancel a goal, and tell SUCCEEDED, CANCELED and ABORTED apart?
- ☐ say when a job should be an action instead of a service?
- ☐ turn a camera detection into a point the arm can reach (range/bearing → base_link → tf2 → IK)?
- ☐ move to the next state when something has finished, not after a guessed delay?

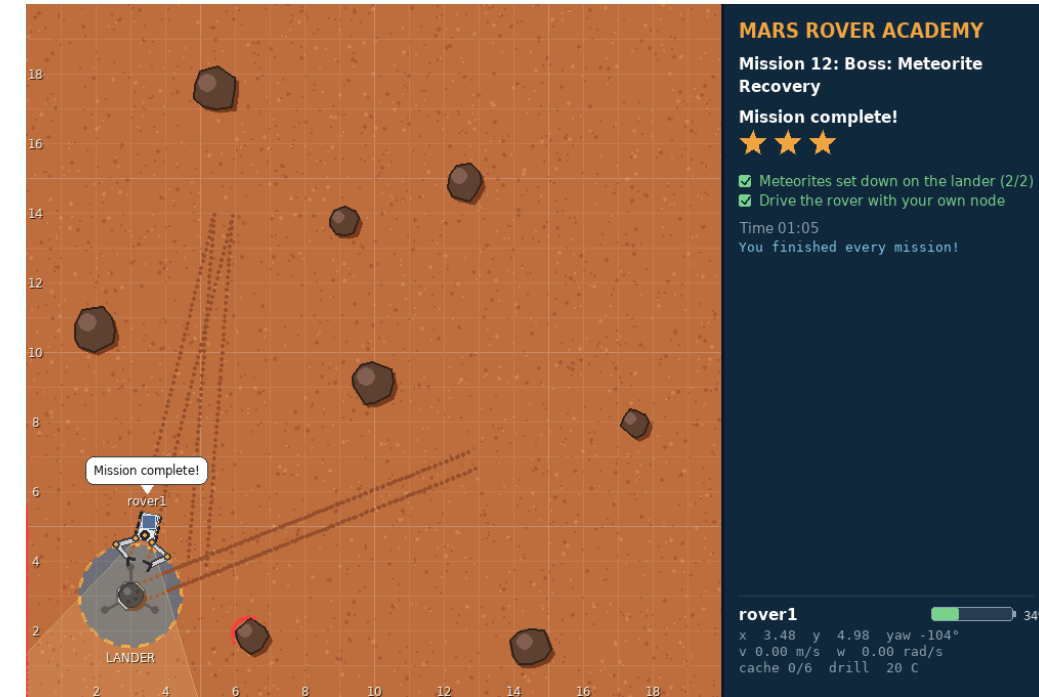
Mission 12 (boss): Meteorite Recovery

You'll learn: mobile manipulation: driving and two-arm coordination together, with a battery to watch

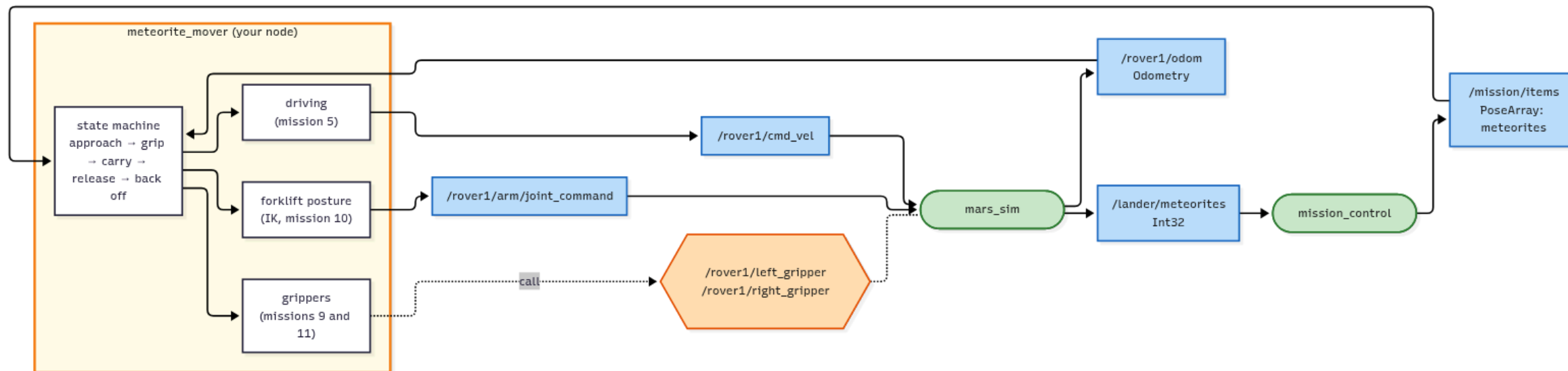
Goal: carry 2 meteorites to the lander. A meteorite only moves when both grippers hold it.

Stars: ≤ 2 min ★★ ★ · ≤ 4 min ★★

✓0 ✓1 ✓2 ✓3 ✓4 ✓5 ✓6 ✓7 ✓8 ✓9 ✓10 ✓11 ►12



Driving and arms together



- One state machine on top: approach, grip, carry, release, back off
- Driving goes out on `cmd_vel`, the arms on `arm/joint_command`, the grippers as service calls
- With the arms fixed, the hard part is a driving problem you already solved in mission 5

Run it

Copy the skeleton from the guide to `src/my_rover/my_rover/meteorite_mover.py`, add `'meteorite_mover = my_rover.meteorite_mover:main'`, to `setup.py`, then:

```
cd ~/mars_rover
colcon build --symlink-install --packages-select my_rover
source install/setup.bash
ros2 launch mission_control mission.launch.py mission:=12      # terminal 1
ros2 run my_rover meteorite_mover                             # terminal 2
```

Meteorites on `/mission/items` · battery on `/rover1/battery` (starts at 60 %, drains 3 times faster).

Build it in stages: the posture, then parking, then gripping, then carrying.

The forklift posture

```
if self.posture is None:
    angles = []
    for side, y in (('left', GRIP_Y), ('right', -GRIP_Y)):
        sx, sy = SHOULDER[side]
        angles += inverse_kinematics(REACH - sx, y - sy, side)
    self.posture = JointState()
    self.posture.name = ['left_shoulder', 'left_elbow', 'right_shoulder', 'right_elbow']
    self.posture.position = angles
    self.arm_pub.publish(self.posture)
```

`REACH` = 1.0 m in front, `GRIP_Y` = 0.35 m either side: computed once with mission 10's IK.

Can you...

- ☐ run two control loops (driving and arms) from one state machine?
- ☐ turn a manipulation problem into a driving problem with a fixed arm posture?
- ☐ explain why both grippers must hold the meteorite, and what happens if they drift apart?
- ☐ sketch how you would split `meteorite_mover` into several nodes?

Course complete

Keep track in **CHECKLIST.md** · every part of ROS 2 and what to learn next in
CONCEPTS.md · how the parts fit together in **ARCHITECTURE.md**

`ros2 run mission_control progress` : how many of the 39 stars did you get?